Vue.js 3 Cheat Sheet & Architecture

COMPOSITION API + SCRIPT SETUP

Guía de Referencia Esencial para Nivel Senior / Experto

1. Fundamentos Arquitectónicos y Reactividad Avanzada

Sistemas de Reactividad: ref() vs reactive()

- `ref(val)` : Para primitivos (o cualquier valor). Envuelve el valor en un objeto con propiedad `.value` . Es el estándar recomendado.
- `reactive(obj)` : Solo para Objetos/Arrays. Usa *Proxy* directamente. **Cuidado:** Pierde reactividad si se desestructura o se reasigna.
- `shallowRef()` / `shallowReactive()` : Reactividad solo en la raíz (optimización masiva de performance).
- `toRefs()` / `toRef()` : Convierte propiedades de un objeto reactivo en refs individuales sin perder el vínculo de reactividad.

Calculados y Obseravadores (Computed & Watchers)

- `computed(() => val)` : Valores derivados con **memoización**. Solo se re-evalúan si cambian sus dependencias reactivas.
- `watch(source, callback)` : Control perezoso (lazy) de efectos secundarios al cambiar un estado específico.
- `watchEffect()` : Ejecuta inmediatamente la función y rastrea *automáticamente* cualquier dependencia usada en su interior.

2. Arquitectura de Componentes y Comunicación

Comunicación Directa & Props

- `defineProps<Props>()` : Declaración de Props con TypeScript nativo sin runtime overhead.
- `defineEmits<Emits>()` : Tipado estricto de eventos enviados al padre.
- `defineModel()` : (Vue 3.4+) Reemplaza la sintaxis verbosa de `v-model` . Maneja binding bidireccional automáticamente.
- `defineExpose()` : Expone explícitamente métodos/ propiedades públicas cuando se usa `ref` sobre el componente.

Patrones Avanzados y Reusabilidad

- **Composables (Custom Hooks):** Funciones que encapsulan lógica con estado reactivo (reemplazo absoluto de Mixins).
- **Provide / Inject:** Inyección de dependencias a cualquier nivel del árbol de componentes (evita *prop-drilling*).
- **Slots (Default, Named, Scoped):** Inyección de plantilla flexible con paso de datos desde el hijo hacia el padre.
- **Teleport:** Renderiza un sub-árbol DOM fuera de la jerarquía actual (ideal para modales, tooltips).

3. Optimización de Rendimiento y Ecosistema Oficial

Diferenciadores Clave para Entrevistas/Nivel Senior: Vue 3 usa un Virtual DOM optimizado con *Compiler-Informed Virtual DOM* (Block Tree, Hoisting de Nodos Estáticos y Flags Parcheables) que minimiza las comparaciones en tiempo de ejecución a casi cero en contenido estático.

Ecosistema Primario

- **Pinia:** El Store Oficial (reemplazo de Vuex). Modular, sin mutaciones complejas, Soporte TS nativo y muy ligero (<1kb).
- **Vue Router 4:** Rutas dinámicas, Guards de navegación globales/por ruta y carga perezosa de componentes via `import()`.
- **Vue Use:** Colección indispensable de utilidades de Composition API (composables de comunidad).

Técnicas de Optimización

- `defineAsyncComponent()`: Code splitting y Lazy Loading de componentes pesados.
- `<KeepAlive>`: Mantiene en memoria instancias de componentes para evitar re-renders costosos.
- `v-once` & `v-memo`: Omite re-evaluaciones del VDOM basándose en condiciones controladas.

4. Ejercicio Práctico Integrado: Single File Component (<script setup lang="ts">)

```
<script setup lang="ts">
import { ref, computed, onMounted } from 'vue';

// 1. Props y Emits Tipados
const props = defineProps<{ initialLimit?: number }>();
const emit = defineEmits<{ (e: 'itemSelected', id: number): void }>();

// 2. Estado Reactivo & Computed
const items = ref<Array<{ id: number; name: string }>>([]);
const filter = ref('');

const filteredItems = computed(() => {
  return items.value
    .filter(i => i.name.toLowerCase().includes(filter.value.toLowerCase()))
    .slice(0, props.initialLimit ?? 10);
});

// 3. Métodos y Lifecycle Hooks
const selectItem = (id: number) => emit('itemSelected', id);

onMounted(async () => {
  // Simulación de Fetch a API
  items.value = [
    { id: 1, name: 'Composition API' },
    { id: 2, name: 'Pinia State' },
    { id: 3, name: 'Vue Router Guards' }
  ];
});
</script>

<template>
  <div class="search-box">
    <input v-model="filter" placeholder="Filtrar conceptos de Vue 3..." />
    <ul v-if="filteredItems.length">
      <li v-for="item in filteredItems" :key="item.id" @click="selectItem(item.id)">
        {{ item.name }}
      </li>
    </ul>
    <p v-else>No se encontraron resultados.</p>
  </div>
</template>
```